

# HPS CUSTOM PRINT & DESIGN

## Project Status Report and Next Build Plan

Prepared from the current implementation state

Report date: 29 September 2026

| Overall roadmap completion    | Approximately 28%                                                                     |
|-------------------------------|---------------------------------------------------------------------------------------|
| Frontend prototype foundation | Approximately 85%                                                                     |
| Backend foundation            | Approximately 25%                                                                     |
| Production readiness          | Not ready yet - core database, payment, admin, production and deployment work remains |
| Current storage mode          | JSON development fallback; PostgreSQL schema and connection tooling prepared          |

### Executive summary

The project has progressed from a single-page prototype into a structured custom-print commerce foundation. The storefront, Design Studio, cart, server-side quote calculation, order API, authentication, customer order access and PostgreSQL migration design are now present. The next major transition is replacing the JSON development stores with PostgreSQL repositories and then building the operational admin, payment and production workflows.

# Section 1 - What has been built

This section records the implementation completed so far, the current architecture, the files created and the behavior that has been verified.

## 1.1 Frontend and UI/UX

- HPS brand system with responsive colors, typography, spacing, radii, shadows and button hierarchy.
- Responsive header, desktop navigation, mobile menu, skip link and keyboard focus states.
- Conversion-focused homepage hero with primary design CTA and product discovery CTA.
- Product search, category filters, price filters and live product count.
- Product detail panel connected directly to Design Studio product selection.
- Loading, empty, validation and success messaging patterns for the current flows.

## 1.2 Storefront and product data

- Structured product catalog for T-Shirt, Hoodie, Denim, Bag and Cap.
- Product base prices, categories, descriptions, sizes, colors and print-side rules.
- Product cards are keyboard accessible and support direct customization.
- The frontend catalog shape is aligned with the future database product model.

## 1.3 Design Studio

- Fabric.js canvas with image upload, text, templates, duplicate, delete, color and opacity controls.
- Front and back design state handling.
- Print-area visualization and safe-area guidance.
- Live quality states: Not started, Excellent, Good and Low.
- Artwork boundary and safe-area warnings.
- Device-local save/load preparation using product and side-specific keys.
- Design and product pricing remain connected through printed-side detection.

## 1.4 Cart and checkout

- Cart survives browser refresh using localStorage.
- Quantity increase/decrease and item removal controls.
- Delivery fee display and express production option.
- Bangladesh mobile number validation and required address validation.
- Browser cart sends product, variant, print-side and express data to the API quote endpoint.
- Checkout submits to the backend order endpoint and clears the cart after success.

## 1.5 Backend API

| GET /api/health      | Reports API status and whether PostgreSQL or JSON development storage is active. |
|----------------------|----------------------------------------------------------------------------------|
| GET /api/products    | Returns the authoritative product catalog.                                       |
| POST /api/cart/quote | Validates products and calculates server-side prices.                            |

| POST /api/orders        | Validates checkout, recalculates totals and persists an order. |
|-------------------------|----------------------------------------------------------------|
| POST /api/auth/register | Creates a user with scrypt-hashed password.                    |
| POST /api/auth/login    | Creates an HttpOnly session cookie.                            |
| POST /api/auth/logout   | Invalidates the active session.                                |
| GET /api/auth/me        | Returns the current authenticated user.                        |
| GET /api/orders         | Returns only orders belonging to the authenticated user.       |

## 1.6 Security and persistence foundations

- Server-side price, variant, quantity, print-side and customer validation.
- Password hashing with Node scrypt, random salts and timing-safe comparison.
- HttpOnly session tokens with expiry.
- Authenticated order-history boundary.
- Atomic JSON writes for development order/user storage.
- Explicit JSON development fallback when DATABASE\_URL is not configured.
- Secrets and node\_modules protected by .gitignore.

## 1.7 Database design

The PostgreSQL migration contains 18 tables, 7 indexes and extensive constraints for users, roles, products, variants, print areas, designs, design versions, carts, addresses, orders, payments, production jobs, shipments, reviews, notifications and audit logs. The schema is in database/001\_initial\_schema.sql and is not yet connected to the live repositories.

## 1.8 Current project files

| index.html                      | Current responsive storefront, Design Studio, cart, checkout and account UI. |
|---------------------------------|------------------------------------------------------------------------------|
| server/index.js                 | Node HTTP API, validation, auth, quote and order flows.                      |
| server/db.js                    | Environment-based PostgreSQL pool and health check.                          |
| server/migrate.js               | Transactional migration runner.                                              |
| data/products.json              | Development product source.                                                  |
| data/orders.json                | Development order store.                                                     |
| data/users.json                 | Development user store.                                                      |
| database/001_initial_schema.sql | Initial PostgreSQL relational schema.                                        |
| package.json                    | Node scripts and dependencies.                                               |
| README.md                       | Current implementation and run instructions.                                 |

## Section 2 - What needs to be built next

This section defines what to build next, where it belongs in the project and how it should be implemented professionally.

### 2.1 Immediate next milestone: PostgreSQL repository integration

| Database connection loading | server/db.js                    | Keep Pool configuration environment-based. Add parameterized query helpers and clear connection errors.         |
|-----------------------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Migration execution         | server/migrate.js               | Apply versioned migrations transactionally and record each applied version.                                     |
| Product repository          | server/repositories/products.js | Read products, variants and print areas from PostgreSQL. Keep response shape compatible with GET /api/products. |
| User repository             | server/repositories/users.js    | Move registration, lookup and password-hash persistence from users.json to PostgreSQL.                          |
| Order repository            | server/repositories/orders.js   | Create orders and order items in one transaction. Never trust browser totals.                                   |
| Storage selection           | server/storage.js               | Use PostgreSQL when DATABASE_URL exists; allow JSON fallback only in explicit development mode.                 |
| Seed data                   | database/seed.sql               | Insert categories, products, variants and print-area profiles from the current catalog.                         |

### 2.2 Customer and design persistence

| Saved designs API | server/routes/designs.js                 | Create, list, update, duplicate and delete designs scoped to the authenticated user.         |
|-------------------|------------------------------------------|----------------------------------------------------------------------------------------------|
| Design versioning | database design_versions plus repository | Create immutable versions for every saved revision; attach the exact version to order items. |
| Design export     | server/services/design-export.js         | Generate thumbnails and production files outside the public executable path.                 |
| Addresses         | server/routes/addresses.js               | Store Bangladesh recipient, phone, division, district, area and address records per user.    |
| Cart ownership    | server/repositories/carts.js             | Merge anonymous cart into authenticated cart on login, then persist cart items.              |

### 2.3 Checkout and payment

| Checkout stepper     | index.html, later apps/web       | Separate Customer, Delivery, Payment, Review and Confirm steps. Keep payable total visible.  |
|----------------------|----------------------------------|----------------------------------------------------------------------------------------------|
| Delivery rules       | server/services/pricing.js       | Calculate configurable delivery zones and fees on the server.                                |
| Payment adapters     | server/payments/                 | Use provider-specific adapters for COD, bKash, Nagad and card gateway.                       |
| Webhook verification | server/routes/webhooks.js        | Verify signatures, reject duplicates, store provider references and reconcile payment state. |
| Idempotency          | server/middleware/idempotency.js | Require an idempotency key for order/payment creation to prevent duplicate orders.           |

## 2.4 Admin and production operations

| Admin application     | apps/admin/                                | Create a separate authenticated admin surface; do not expose staff operations in the customer page. |
|-----------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Roles and permissions | server/auth/authorization.js               | Enforce customer, staff and admin permissions at every route.                                       |
| Order management      | apps/admin/orders/                         | Search orders, verify payments, inspect designs and update statuses.                                |
| Design review         | apps/admin/designs/                        | Approve or reject with a required reason; allow customer revision/resubmission.                     |
| Production queue      | apps/admin/production/                     | Implement received, approved, printing, QC and packaging states with audit events.                  |
| Shipment management   | server/routes/shipments.js and apps/admin/ | Create shipment records, tracking numbers and customer-visible status.                              |

## 2.5 Secure storage and notifications

- Create an object-storage adapter for product images, user designs, thumbnails, templates, print files, invoices and order files.
- Validate MIME type, extension, size and image/SVG content before storage.
- Use randomized object names and private storage paths.
- Add notification jobs for order confirmation, payment confirmation, design review, approval, rejection, printing, packaging, shipment and delivery.
- Start with email/web notifications; add SMS or WhatsApp after operational requirements are clear.

## 2.6 Testing and launch hardening

- Add API tests for quote validation, duplicate callbacks, authorization isolation and order transactions.
- Add Design Studio tests for upload, text, resize, rotation, front/back, save/load and export.
- Add upload-abuse tests, SVG sanitization tests and rate-limit tests.
- Run database backup and restore tests.
- Run responsive, accessibility, browser and performance checks.
- Configure production HTTPS, secrets, logs, monitoring, alerts, rollback and controlled launch.

## 2.7 Recommended build order

| 1 | PostgreSQL repositories and seed data | Products, users and orders persist in the relational database.                     |
|---|---------------------------------------|------------------------------------------------------------------------------------|
| 2 | Design and address persistence        | Customers can save designs and addresses securely.                                 |
| 3 | Checkout hardening                    | Server quote, cart merge, delivery zones and idempotent order creation work.       |
| 4 | Payment integration                   | At least COD plus one verified online method works in sandbox.                     |
| 5 | Admin dashboard                       | Staff can operate products, orders, designs and customers without database access. |
| 6 | Production and shipment               | Approved designs move through printing, QC, packaging and shipment.                |
| 7 | Security and QA                       | Security, backup, accessibility and regression tests pass.                         |

|   |            |                                                     |
|---|------------|-----------------------------------------------------|
|   |            |                                                     |
| 8 | Deployment | HTTPS, monitoring, backups and rollback are active. |

## Conclusion

The project is progressing in the correct order: user experience first, then authoritative business rules, then persistence and operational workflows. The immediate priority is PostgreSQL repository integration. Once that is complete, the project will have a reliable data core for customer accounts, designs, orders, payments, production and delivery.